```
object HttpServerShardedContainer {
  ...|

    get {          ❶
      parameters("entityId".as[String]) { entityId =>
        val container = sharding.entityRefFor(Container.TypeKey, entityId)

        val response: Future[Container.Cargos] =
           container
             .ask(Container.GetCargos)
             .mapTo[Container.Cargos]

        complete(response)
       }
    })
  ...
}
```

❶ This get is inside a path(“cargo”).

With this last part, you have everything you need to create the Akka application with an Akka HTTP server as the port.

Listing 11.4 Akka application with ports to `Container`

```
object HttpServerShardedContainer {

  def main(args: Array[String]): Unit = {
    implicit val system = ActorSystem(Behaviors.empty, "simple-api")

    implicit val executionContext = system.executionContext              ❶

    implicit val timeout: Timeout = 3.seconds                            ❷

    implicit val cargoFormat: JsonFormat[Container.Cargo] = ...          ❸
    implicit val cargosFormat: RootJsonFormat[Container.Cargos] = ...    ❸

    val sharding = ClusterSharding(system)

    val shardingRegion =
      sharding.init(Entity(Container.TypeKey)(entityContext =>
        Container(entityContext.entityId)))

    val route: Route =                                                   ❹
      path("cargo") {
```